

TechTonics

Painel de Gestão, Engajamento e Fidelização

Cannoli Foodtech

Integrantes

Fernanda Loura da Silva
Gustavo Henrique da Silva Santos
Lucas Alves Bernardo
Nicolly da Silva Soares
Thiffany Moraes Vieira da Silva

Professor Orientador PI:

Eduardo Savino Gomes
Ciência de Dados

Professores Orientadores complementares:

Lucy Mari Tabuti
Engenharia de Software e Arquitetura de Sistemas

Maurício Lopes da Cunha
Contabilidade e Finanças

Rodnil da Silva Moreira Lisboa
Análise Inferencial de Dados

Resumo Executivo

Este relatório documenta a **Entrega 2 de Engenharia de Software** do Projeto Integrador do 4º Semestre, elaborado pelo grupo **TechTonics**. O foco desta entrega é a **aplicação de design de software** e a elaboração de **diagramas UML** sobre a implementação real do sistema, descrevendo as entradas e saídas e apresentando as observações reflexivas do grupo.

O produto desenvolvido é o **Canolli360** — um painel analítico (BI) web construído em **Python com Streamlit** que lê bases tabulares (CSV), filtra por período e loja, calcula KPIs financeiros e operacionais e exibe dashboards interativos.

1. Visão Geral da Arquitetura

1.1. Visão Geral da Arquitetura — Separação em Camadas

O projeto organiza as responsabilidades em módulos distintos, aplicando o princípio de **Separação de Interesses (Separation of Concerns)** e uma arquitetura lógica em três camadas: **Dados → Lógica → Apresentação**. Cada arquivo Python possui um papel bem definido:

Módulo	Camada	Responsabilidade
app.py	Apresentação / Orquestração	Ponto de entrada principal: carrega dados via <code>@st.cache_data</code> , chama menu, aplica filtros, dispara cálculos e monta o layout (cards + gráfico de receita).
menu.py	Apresentação / Navegação	Header com filtros de período e loja (<code>st.session_state</code>); sidebar com mapa de páginas (PAGINAS); lógica de troca de página via <code>st.switch_page</code> .
funcs.py	Lógica de Negócio	Camada de domínio reutilizável: filtros temporais, agregações (receita, pedidos, ticket médio, clientes ativos), série mensal e geração de gráfico Altair.
cards.py	Apresentação / UI	Componentes visuais de métricas renderizados em HTML/CSS embutido. Inclui <code>card_valor()</code> e <code>card_pizza5()</code> para top-5 campanhas por loja.
pages/indicadores.py	Apresentação / Análise	Página analítica estendida: decomposição de receita, cancelamentos, eficiência operacional, concentração (HHI, Gini), promoções e crescimento.
graficos.py	Lógica / Visualização	Helpers de visualização Altair compartilháveis entre páginas.

tabelas.py	Dados / Acesso	Leitura alternativa de CSVs (caminhos próprios). Atenção: divergência de paths com app.py — ponto de melhoria identificado.
-------------------	----------------	---

1.2. Padrão de Fluxo de Dados — Pipeline Linear

A arquitetura implementa um **padrão pipeline** de processamento de dados, onde cada etapa recebe o resultado da anterior como entrada:

Etapa	Componente	Transformação
1 — Ingestão	app.py + @st.cache_data	Leitura dos 6 CSVs em memória; cache evita releitura a cada interação do usuário.
2 — Filtro	menu.py → funcs.filtrar_periodo	Subconjunto de StoreOrder por loja (storeid) e por período (createdat ≥ data_inicio).
3 — Cálculo	funcs.py	KPIs: receita_total, total_pedidos, ticket_medio, clientes_ativos calculados sobre df_filtrado.
4 — Formatação	funcs.formatar_moeda	Conversão de floats para string monetária no padrão brasileiro (R\$ X.XXX,XX).
5 — Exibição	cards.py + app.py + graficos.py	Cards HTML/CSS + gráfico Altair de linha (receita por mês) renderizados no Streamlit.

1.3. Decisões de Design Relevantes

- **Cache de dados (@st.cache_data):** a função carregar_dados() em app.py usa cache do Streamlit para não reler os 6 CSVs a cada interação — trade-off entre uso de memória e desempenho de resposta.
- **Estado de sessão (st.session_state):** os filtros de período e loja são armazenados em session_state, preservando a seleção do usuário entre recargas da página.
- **Filtros como contexto global:** o par (periodo, restaurante) funciona como contexto compartilhado que produz df_filtrado — entrada única para todos os KPIs subsequentes.
- **HTML/CSS embutido nos cards:** cards.py usa st.markdown com unsafe_allow_html=True para estilização avançada dos cards de métricas, evitando dependências de bibliotecas externas de UI.

Separação Altair em graficos.py: os helpers de visualização foram isolados em módulo próprio, permitindo reuso entre as páginas Finance e Indicadores.

2. Entradas do Sistema

2.1. Fontes de Dados (Arquivos CSV)

A tabela abaixo consolida os 11 requisitos funcionais, unindo os requisitos técnicos do dashboard com os requisitos de negócio relacionados às campanhas promocionais:

Arquivo CSV	Entidade	Colunas-chave utilizadas
STORE.csv	Restaurante	id, name, status
CUSTOMER.csv	Cliente	id, status, email (hash LGPD)
STOREORDER.csv	Pedido	id, storeid, customerid, status (16=concluído), totalamount, subtotalamount, discountamount, taxamount, createdat, scheduledat
CUSTOMERADDRESS.csv	Endereço	id, customerid, city
CAMPAIGN.csv	Campanha	id, storeid, templateid
CAMPAIGNxORDER.csv	Campanha×Pedido	id, campaignid, orderid, storeid, message_id

2.1.2 Entradas de Interação do Usuário

Controle UI	Componente	Valores possíveis	Efeito
Filtro Período	menu.py → selectbox	Última semana / mês / bimestre / trimestre / semestre / ano / Todo o período	Define data_inicio via _calcular_inicio(); filtra createdat em func.filtrar_periodo.
Filtro Loja	menu.py → selectbox	"Todas" ou nome de restaurante da lista	Filtra StoreOrder por storeid correspondente ao nome selecionado.
Navegação	menu.py → render_sidebar	Finance / Indicadores / Retention* / Settings*	Troca a página ativa via st.switch_page(PAGINAS[menu]).

2.2. Saídas do Sistema

2.2.1 Painel Finance (app.py) — KPIs Principais

KPI Exibido	Fórmula / Lógica	Função
Receita Total	SUM(totalamount) WHERE status = 16	funcs.receita_total()
Total de Vendas	COUNT(id) WHERE status = 16	funcs.total_pedidos()

Ticket Médio	Receita Total ÷ Total de Vendas	funcs.ticket_medio()
Clientes Ativos	DISTINCT customerid com status=1, no período	funcs.clientes_ativos()
Receita por Mês	Resample mensal de totalamount (status=16)	funcs.valor_per_mes() + Altair

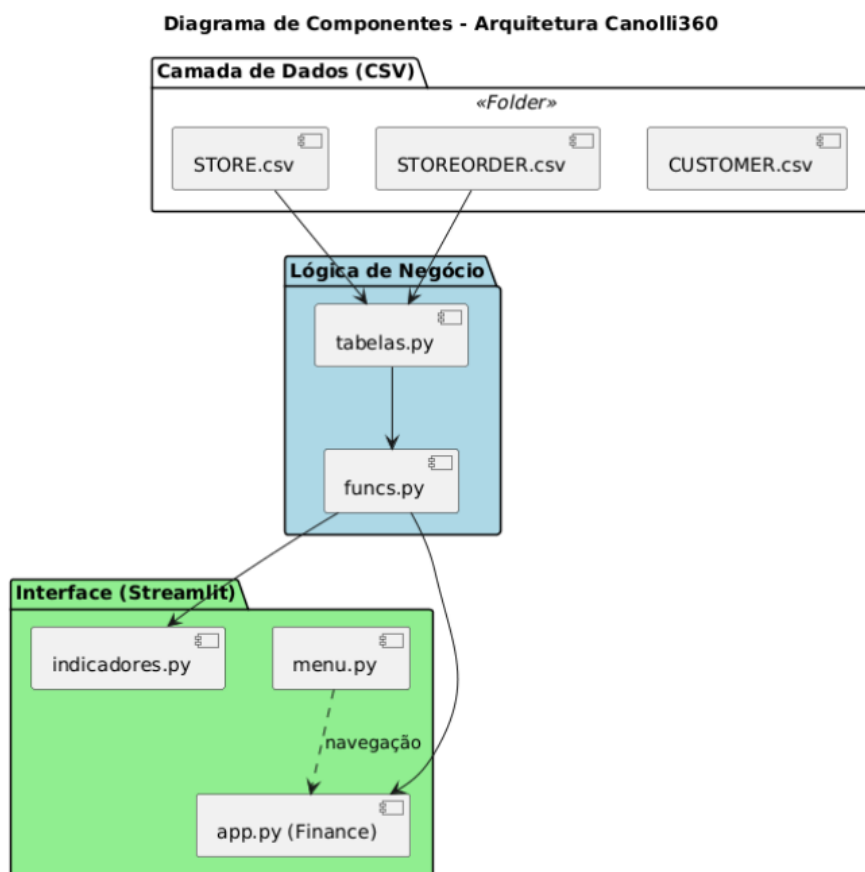
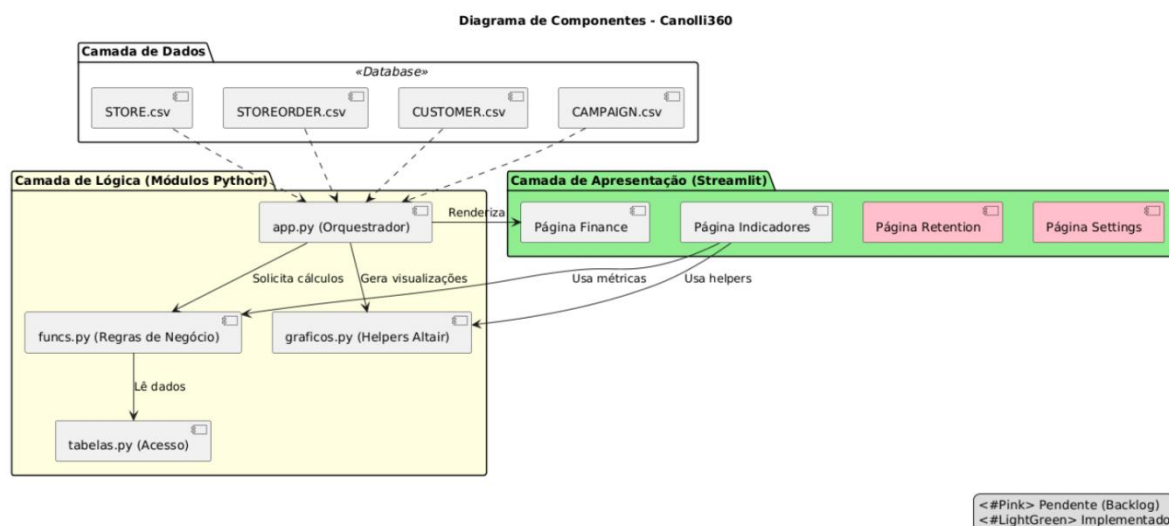
2.2.2 Página Indicadores (pages/indicadores.py) — Análise Aprofundada

Seção	Métricas Calculadas
1.0 Estrutura de Receita	Subtotal, descontos, impostos, receita total; % sobre receita; Receita Líquida Comercial; Taxa de Realização; Custo de Oportunidade dos não-concluídos.
2.0 Cancelamento	Taxa de cancelamento efetivo (status 8, 11, 14); cancelamentos vs. universo total.
3.0 Produtividade	Pedidos por loja, receita por loja, eficiência relativa.
4.0 Concentração	HHI (Herfindahl-Hirschman Index) e coeficiente de Gini para concentração de receita por loja.
5.0 Promoções	Receita com e sem desconto; impacto percentual de promoções na margem.
6.0 Crescimento	Variação percentual de receita entre períodos consecutivos.

3. Diagramas

3.1. Diagrama de Componentes

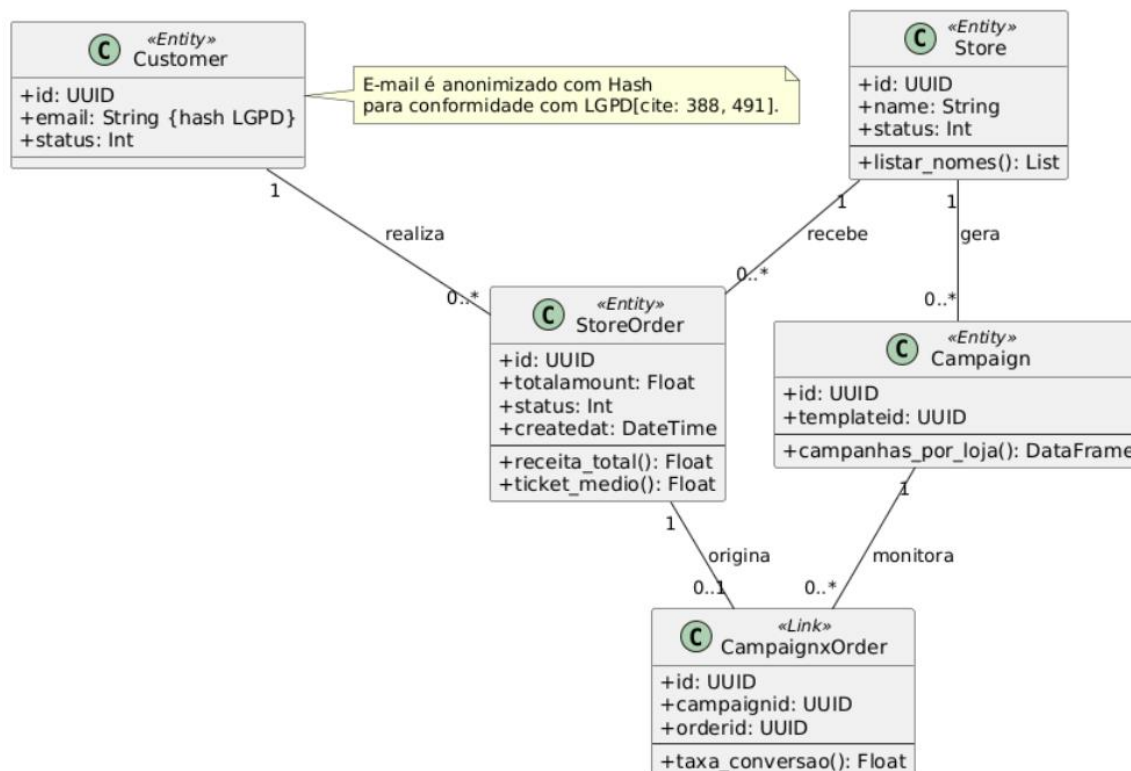
O diagrama abaixo representa a **organização arquitetural em camadas** do sistema Canolli360, ilustrando os módulos Python, suas responsabilidades e as dependências entre as camadas de Dados, Lógica e Apresentação. Módulos em vermelho indicam páginas referenciadas no menu mas ainda não implementadas no repositório atual.



3.2. Diagrama de Classes

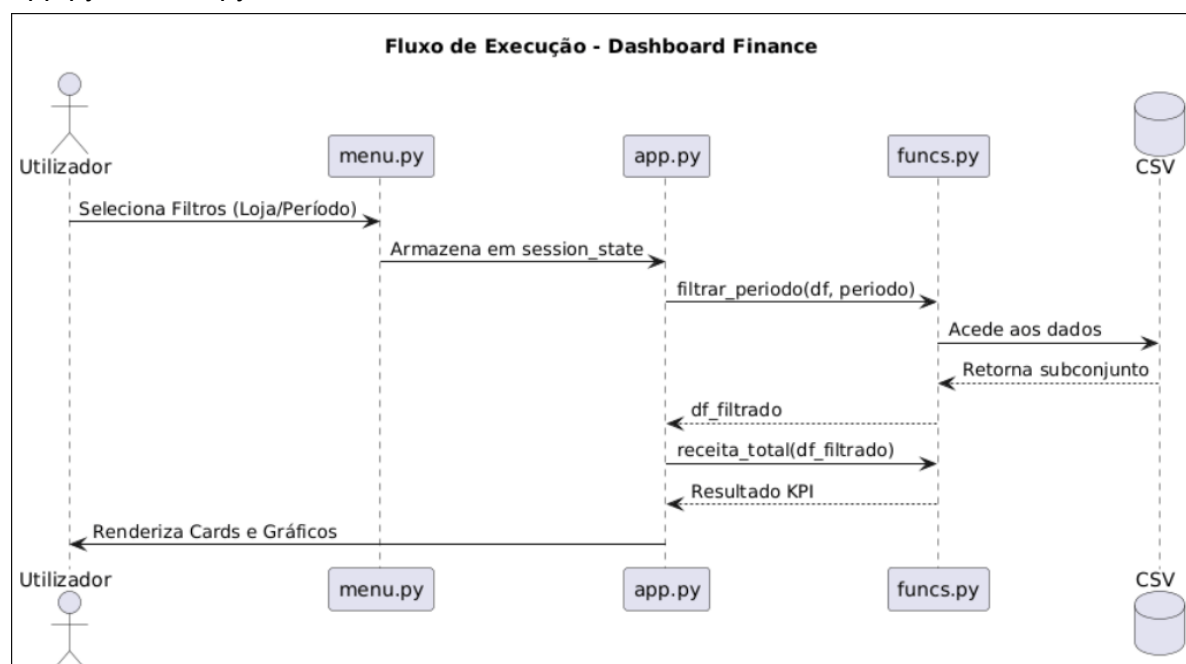
O diagrama de classes modela as **entidades do domínio** do sistema, derivadas dos arquivos CSV fornecidos pela Cannoli. Estão representados os atributos principais, os métodos de negócio implementados em funcs.py e os relacionamentos entre as entidades (associações 1:N e dependências).

Diagrama de Classes - Entidades do Domínio Cannoli360



3.3. Diagrama de Sequência

O diagrama de sequência ilustra o **fluxo principal de interação** entre o usuário e os componentes do sistema: desde a seleção de filtros até a exibição dos cards de KPI e do gráfico de receita mensal. O diagrama evidencia a cadeia de chamadas entre `menu.py` → `app.py` → `funcs.py` → `CSV`.



4. Reflexões e Observações do Grupo

4.1. O que funcionou bem

- **Separação de responsabilidades em funcs.py:** centralizar a lógica de KPIs em um único módulo facilitou a reutilização entre app.py e indicadores.py, além de simplificar os testes manuais das métricas.
- **@st.cache_data em carregar_dados():** a decisão de cachear a leitura dos CSVs foi acertada — eliminou a lentidão de releitura a cada interação e permitiu trabalhar com os 118.936 pedidos reais sem degradação perceptível.
- **st.session_state nos filtros:** a gestão de estado via session_state evitou perda de seleção ao navegar entre páginas, melhorando a experiência do usuário.
- **Pair Programming na modelagem:** trabalhar em duplas na definição das fórmulas de KPI

(especialmente HHI e Gini) reduziu erros de cálculo e acelerou a revisão dos resultados.

4.2. Divergências entre Design e Implementação

Durante o desenvolvimento, o grupo identificou **divergências entre o design planejado e a implementação atual** — oportunidades de melhoria para as próximas sprints:

Divergência	Situação Atual	Ação Prevista
Páginas Retention e Settings referenciadas no menu mas não implementadas	menu.PAGINAS aponta para pages/Retention.py e pages/Settings.py inexistentes — gera erro em runtime se clicadas.	Criar stubs das páginas ou remover do menu até implementação.
tabelas.py com paths divergentes de app.py	tabelas.py usa caminhos de CSV diferentes dos usados em app.py e indicadores.py — risco de inconsistência de dados.	Centralizar leitura de dados em uma única função (DRY), preferencialmente em funcs.py.
Lógica de negócio dispersa em indicadores.py	Parte dos cálculos pesados (HHI, Gini, cancelamentos) está inline em indicadores.py em vez de funcs.py.	Mover regras de negócio para funcs.py, deixando indicadores.py apenas com chamadas e renderização.
Ausência de testes automatizados	Não há testes unitários para as funções de funcs.py — métricas validadas apenas manualmente.	Implementar pytest com casos de teste para as funções críticas de KPI.

4.3. Aprendizados e Próximos Passos

O desenvolvimento do Canolli360 consolidou para o grupo a compreensão prática de que **design de software não é etapa inicial isolada** — é um processo contínuo que deve ser revisado a cada sprint conforme a complexidade dos dados e dos requisitos evolui.

- **ETL antes de tudo:** a qualidade dos CSVs da Cannoli determinou o escopo de todo o projeto; a próxima sprint priorizará o log de erros de ingestão (RF12).
- **Modularização incremental:** mover a lógica de indicadores.py para funcs.py será feito gradualmente, garantindo que os resultados atuais não regridam.
- **Supabase (PostgreSQL):** a migração dos CSVs para banco relacional em nuvem (RNF06) está planejada para eliminar os problemas de paths divergentes e suportar atualização em tempo real.

Testes automatizados: implementar pytest antes da Semana FECAP de Tecnologia para garantir estabilidade das métricas durante a apresentação.